Fleetbase Orchestrator: Technical Implementation Plan

1. Database Migrations & Schema Updates

To support advanced Vehicle Routing Problem (VRP) constraints like capacities, time windows, and skills, several core FleetOps models require schema updates.

1.1. `vehicles` Table

Vehicles need to define their capacities and the skills/equipment they possess.

- **Add `skills` (JSON):** An array of strings representing capabilities (e.g., `["refrigerated", "tail-lift"]`).
- **Add `time_window_start` / `time_window_end` (Time):** The default working hours for the vehicle/driver.
- **Add `max_tasks` (Integer):** Maximum number of stops the vehicle can handle in one route.

1.2. `drivers` Table

Drivers need specific skills and working hour constraints.

- **Add `skills` (JSON):** An array of strings representing driver qualifications (e.g., `["hazmat", "forklift"]`).
- **Add `max_travel_time` (Integer):** Maximum allowed driving time in seconds.
- **Add `max_distance` (Integer):** Maximum allowed driving distance in meters.

1.3. `orders` Table

Orders need to define their requirements and time constraints.

- **Add `time_window_start` / `time_window_end` (DateTime):** The strict or soft time window for the overall order completion.
- **Add `required_skills` (JSON):** An array of strings representing skills required to service this order.
- **Add `priority` (Integer):** A routing priority score (e.g., 1-100) used by the optimization engine.

1.4. `payloads` & `entities` Tables

The `entities` table already has `weight`, `length`, `width`, and `height`. The `payloads` table should aggregate these or define overall capacity requirements.

- **Add `capacity_requirements` (JSON) to `payloads`:** A normalized object representing the multi-dimensional capacity required (e.g., `{"weight": 500, "volume": 20}`).

1.5. `waypoints` Table

Waypoints need specific time windows for pickups vs. dropoffs.

- **Add `time_window_start` / `time_window_end` (DateTime):** Specific time windows for this exact stop.
- **Add `service_time` (Integer):** Expected duration in seconds to complete the task at this waypoint (e.g., 600s for a 10-minute dropoff).

2. Backend Architecture Updates

2.1. `AllocationController` Refactoring

The `AllocationController` currently handles basic assignment. It must be expanded to support the new constraints.

- **Update `run()`:** The payload sent to the `AllocationEngineInterface` must now include the new constraint fields (skills, time windows, capacities) extracted from the models.

- **Update `commit()`:** Ensure that when routes are committed, the sequence of waypoints is properly saved to the database, not just the `driver_assigned_uuid`.

2.2. `VroomAllocationEngine` PHP Adapter

The VROOM adapter must translate Fleetbase models into the specific JSON schema expected by the VROOM API.

- **Map Capacities:** Translate `vehicle.payload_capacity` and `entity.weight` into VROOM's `capacity` and `delivery` arrays.
- **Map Time Windows:** Translate `time_window_start` / `end` into VROOM's `time_windows` array (Unix timestamps).
- **Map Skills:** Translate `required_skills` and `vehicle.skills` into VROOM's `skills` array (integer IDs mapped from strings).

3. Frontend Architecture Updates

3.1. `OrchestratorWorkbench` Component

The workbench must be refactored from a simple list view into the 3-panel layout described in the UI/UX plan.

- **State Management:** The component needs robust state management to handle the transition between "Allocation Mode" (pre-run) and "Route Review Mode" (post-run).
- **Map Integration:** Integrate `ember-leaflet` or similar to render order pins, depot markers, and route polylines.
- **Timeline Integration:** Implement a Gantt-chart component (e.g., using D3.js or a dedicated Ember addon) to visualize routes over time.

3.2. Order Import Flow

- **New Component: `OrderImportModal`:** A modal handling the 3-step import process (Upload -> Map Columns -> Validate).

- **CSV Parsing:** Use a library like `PapaParse` for client-side CSV parsing.
- **Geocoding Service:** Integrate with the existing Fleetbase geocoding services to validate addresses during step 3 of the import flow.

3.3. Drag-and-Drop & Interactivity

- **Drag-and-Drop:** Use `ember-drag-drop` to allow dragging order cards from the pool onto the timeline or map.
- **Action Handlers:** Implement actions for `onOrderDropped`, `onRouteResequenced`, and `onLassoSelection`.

4. Phased Implementation Plan

1. **Database & Models:** Create and run migrations for the new constraint columns. Update the Eloquent models to make these fields fillable and cast JSON columns appropriately.
2. **Backend Engine Adapter:** Update the `VroomAllocationEngine` to serialize the new constraints into the VROOM API payload.
3. **Frontend Import Flow:** Build the `OrderImportModal` and integrate it into the workbench toolbar.
4. **Frontend Workbench UI:** Refactor the `OrchestratorWorkbench` into the 3-panel layout. Implement the map and timeline views.
5. **Interactivity:** Add drag-and-drop routing, lasso selection, and route resequencing capabilities to the frontend.